

SituationReport

testdata_generator

Benutzerhandbuch

Synthetische Jira-Daten erzeugen und echte Daten aus Jira Cloud exportieren

situation-report -- github.com/Jaegerfeld/situation-report

Fuer Agile Coaches und PI Manager -- Version 0.9.4

1 Was ist der Testdata Generator?

Der **Testdata Generator** erzeugt synthetische Jira-Issue-Daten im Jira-REST-API-Format. Die generierten Dateien sind direkt mit dem Modul **transform_data** verarbeitbar — ohne eine echte Jira-Instanz.

Der Generator simuliert realistische Workflow-Historien: Issues durchlaufen die definierten Stages, verbleiben unterschiedlich lange in jeder Stage, werden gelegentlich zurückgestuft und schließen mit konfigurierbarer Rate ab.

Typische Einsatzgebiete:

- **Neue Installation testen:** Prüfen, ob transform_data und build_reports korrekt arbeiten
- **Demos und Präsentationen:** Realistische Daten ohne Datenschutzbedenken
- **Schulungen:** Lernumgebung mit bekannten, kontrollierten Daten
- **Entwicklung:** Reproduzierbare Testdaten über den Seed-Parameter

Der Testdata Generator ergänzt das Modul **Get Data** (noch in Entwicklung), das echte Daten direkt aus Jira lädt. Für Produktiv-umgebungen mit echten Jira-Daten siehe Kapitel 6.

2 Voraussetzungen und Start

2.1 Portables Paket (empfohlen)

Das portable Paket enthält Python bereits eingebettet — keine separate Installation erforderlich.

Betriebssystem	Startdatei
Windows	TestdataGenerator.bat (Doppelklick)
macOS	TestdataGenerator.command (Rechtsklick → Öffnen)
Linux	./TestdataGenerator.sh

Hinweis für Windows: Falls beim Start eine Fehlermeldung erscheint, die Python nicht findet — die ZIP-Datei war möglicherweise durch Windows blockiert. Lösung: Rechtsklick auf die ZIP → Eigenschaften → Sicherheit → Zulassen → OK → erneut entpacken.

2.2 Aus dem Quellcode (Python erforderlich)

```
python -m testdata_generator
```

2.3 Die Benutzeroberfläche

Nach dem Start öffnet sich das Hauptfenster mit Eingabefeldern für alle Parameter. Pflichtfeld ist nur die Workflow-Datei — alle anderen Felder haben sinnvolle Standardwerte.

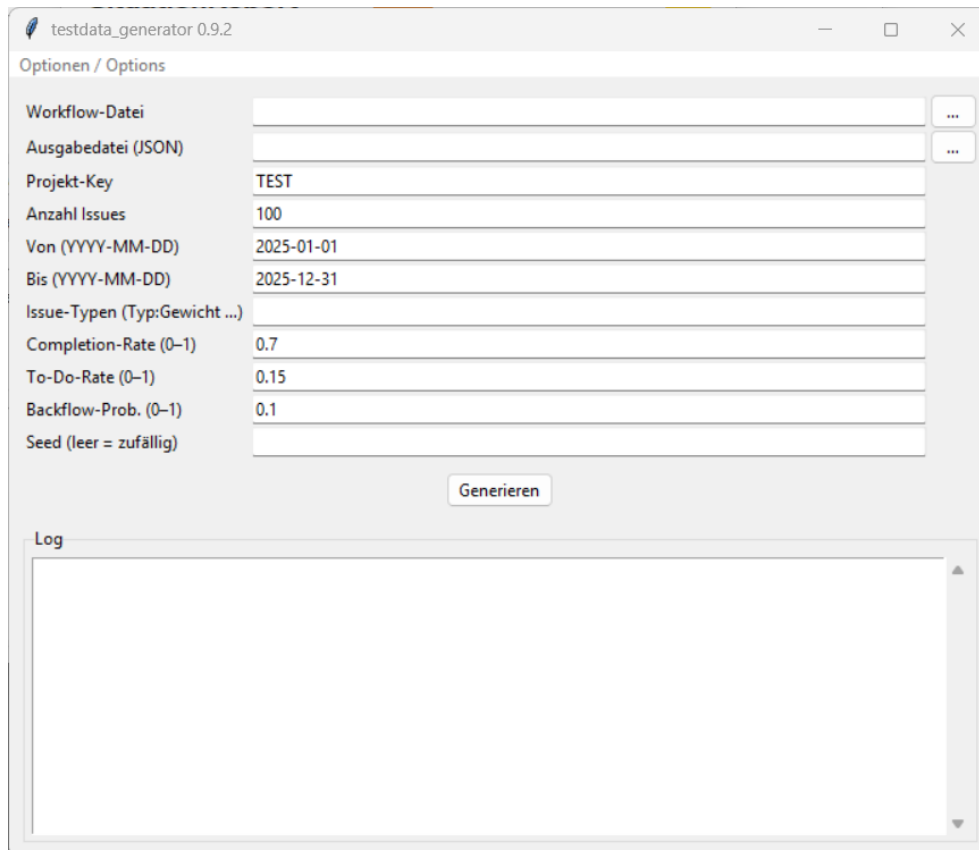


Abb. 1: Benutzeroberfläche des Testdata Generators

3 Die Workflow-Datei

Die Workflow-Datei definiert die Stages Ihres Jira-Boards und ist das einzige Pflichtfeld des Generators. Sie hat dasselbe Format wie in `transform_data` — eine Datei genügt für beide Module.

3.1 Format

Jede Zeile beschreibt eine Stage. Aliases (alternative Status-Namen im Jira-Export) werden durch Doppelpunkte abgetrennt. Zwei Sonderzeilen markieren Anfang und Ende der aktiven Bearbeitung:

```
KanonischerName:Alias1:Alias2
<First>ErsteAktiveStage
<Closed>AbgeschlosseneStage
```

Zeile	Bedeutung
KanonischerName:Alias1:Alias2	Stage mit einem oder mehreren alternativen Jira-Status-Namen
StageName (ohne :)	Stage ohne Aliases
<First>StageName	Erste aktive Stage — wo Bearbeitung beginnt (Cycle-Time-Start)
<Closed>StageName	Abgeschlossene Stage — markiert ein Issue als fertig

3.2 ART_A Beispiel-Workflow

Das mitgelieferte Beispiel `workflow_ART_A.txt` bildet einen typischen SAFe-ART-Workflow ab:

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

Anmerkungen: 'Funnel' enthält vier Aliases — alle vier Jira-Status-Namen werden auf 'Funnel' normiert. Analysis ist die erste aktive Stage (Cycle-Time beginnt hier). Releasing gilt als abgeschlossen. Done (nach Releasing) nimmt auch Canceled-Issues auf.

4 Parameter-Referenz

Alle Parameter sind sowohl in der GUI als auch auf der Kommandozeile verfügbar.

Parameter	Standard	Beschreibung
--workflow FILE	(Pflicht)	Workflow-Definitionsdatei (.txt)
--output FILE.json	<project>_generated.json	Pfad der Ausgabedatei
--project KEY	TEST	Jira-Projekt-Key der generierten Issues
--issues N	100	Anzahl zu generierender Issues
--from-date YYYY-MM-DD	2025-01-01	Frühestes Erstellungsdatum
--to-date YYYY-MM-DD	2025-12-31	Spätestes Übergangsdatum
--issue-types TYPE:W ...	Feature:0.6 Bug:0.3 Enabler:0.1	Issue-Typen mit Gewichtung (Summe muss nicht 1 ergeben)
--completion-rate FLOAT	0.7	Anteil Issues, die die Closed-Stage erreichen (0–1)
--todo-rate FLOAT	0.15	Anteil offener Issues, die in frühen Stages verbleiben (0–1)
--backflow-prob FLOAT	0.1	Wahrscheinlichkeit eines Rückschritts bei jedem Übergang (0–1)
--seed INT	(zufällig)	Seed für reproduzierbare Ausgabe (optional)

Tipp: Reproduzierbare Ergebnisse

Mit einem festen Seed (z. B. --seed 42) erzeugt der Generator bei gleichen Parametern immer dieselbe JSON-Datei — ideal für Demos, bei denen alle Teilnehmer dieselben Ergebnisse sehen sollen.

5 ART_A – Vollständiges Beispiel

Dieses Beispiel zeigt den vollständigen Ablauf vom Generator bis zum fertigen Report am fiktiven Projekt ART_A.

Schritt 1: Testdaten generieren

```
python -m testdata_generator \
  --workflow testdata_generator/workflow_ART_A.txt \
  --project ART_A \
  --issues 200 \
  --from-date 2025-01-01 \
  --to-date 2025-12-31 \
  --issue-types Feature:0.6 Bug:0.2 Enabler:0.2 \
  --completion-rate 0.75 \
  --backflow-prob 0.08 \
  --seed 42 \
  --output ART_A_generated.json
```

Ergebnis: ART_A_generated.json mit 200 Issues, ca. 150 abgeschlossen und 50 in verschiedenen Stages offen. Dank Seed 42 ist die Ausgabe jederzeit reproduzierbar.

Schritt 2: Mit transform_data verarbeiten

```
python -m transform_data ART_A_generated.json \
  --workflow testdata_generator/workflow_ART_A.txt
```

Erzeugt drei Excel-Dateien: ART_A_generated_IssueTimes.xlsx, ART_A_generated_CFD.xlsx und ART_A_generated_Transitions.xlsx.

Schritt 3: Report mit build_reports erstellen

```
python -m build_reports
```

In build_reports die drei Excel-Dateien aus Schritt 2 laden — das Programm erstellt daraus Flow-Diagramme, Cycle-Time-Analysen und Metriken.

Was zu erwarten ist (Seed 42, 200 Issues):

- Issue-Keys: ART_A-0001 bis ART_A-0200
- Issue-Typen: ~120 Feature, ~40 Bug, ~40 Enabler
- Ca. 150 Issues im Status 'Releasing' oder 'Done'
- Erstellungsdaten verteilt über das Jahr 2025
- Gelegentliche Rückschritte (backflow-prob 0.08 = ca. 8 %)

6 Echte Daten aus Jira Cloud exportieren

Dieses Kapitel erklärt, wie Sie echte Issue-Daten aus Jira Cloud für SituationReport exportieren. Der Export liefert dieselbe JSON-Struktur wie der Testdata Generator — transform_data verarbeitet beide identisch.

6.1 Was Sie benötigen

Voraussetzung	Details
Jira Cloud Zugang	Ein Konto mit Lesezugriff auf das gewünschte Projekt
API-Token	Persönliches Token von id.atlassian.com — einmalig erstellen
Projekt-Key	Kürzel des Jira-Projekts, z. B. 'ART_A' oder 'SCRUM'
curl oder Browser	curl für automatisierte Abfragen; Browser für einfache Tests

6.2 API-Token erstellen

Das API-Token ersetzt bei API-Abfragen Ihr Passwort und wird einmalig erstellt:

Schritt	Aktion
1	https://id.atlassian.com im Browser aufrufen (in Jira eingeloggt sein)
2	Security → API tokens → Create API token
3	Einen Namen eingeben, z. B. 'SituationReport', auf 'Create' klicken
4	Token kopieren und sicher speichern — er wird nur einmal angezeigt!

Sicherheitshinweis: Behandeln Sie den API-Token wie ein Passwort. Speichern Sie ihn in einem Passwort-Manager und teilen Sie ihn nicht mit anderen.

6.3 Die Jira REST API abfragen

Die Jira REST API v2 liefert Issues im JSON-Format. Der Parameter expand=changeLog ist **Pflicht** — ohne ihn fehlen die Statusübergänge, die transform_data benötigt.

```
curl -u "name@firma.de:IhrAPIToken" \  
  "https://firma.atlassian.net/rest/api/2/search?\  
  jql=project=ART_A+ORDER+BY+created+ASC\  
  &expand=changeLog\  
  &maxResults=1000\  
  &fields=issuetype,created,status,project,summary,resolution" \  
  -o ART_A_page1.json
```

Ersetzen Sie firma durch Ihre Atlassian-Subdomain und ART_A durch Ihren Projekt-Key.

Browser-Methode (ohne curl)

Wer curl nicht installiert hat, kann die URL direkt im Browser aufrufen — der Browser nutzt die bestehende Jira-Session:


```
https://firma.atlassian.net/rest/api/2/search
?jql=project=ART_A&expand=changelog&maxResults=1000
```

Der Browser zeigt die JSON-Antwort als Text. **Strg+S** → Dateityp 'Alle Dateien' → Als ART_A_page1.json speichern.

6.4 Welche Felder transform_data benötigt

Jira-Feld	Pflicht	Verwendung
key	✓	Issue-Kennung (z. B. ART_A-001)
fields.issuetype.name	✓	Issue-Typ für Filterung (Feature, Bug, ...)
fields.created	✓	Erstellungsdatum des Issues
fields.status.name	✓	Aktueller Status
changelog (expand=changelog)	✓	Statusübergänge mit Zeitstempeln — MUSS im URL-Parameter gesetzt sein
fields.resolution	–	Optional — Auflösungstyp (z. B. 'Done', 'Won't Fix')
fields.summary	–	Optional — Issue-Titel

6.5 Paginierung – mehr als 1.000 Issues

Jira liefert maximal 1.000 Issues pro Anfrage. Bei größeren Projekten mehrere Seiten abrufen und anschließend mit dem Helper zusammenführen:

```
# Seite 1 (Issues 1-1000)
curl -u "name@firma.de:Token" \
  "https://firma.atlassian.net/rest/api/2/search?jql=project=ART_A\
&expand=changelog&maxResults=1000&startAt=0" \
  -o ART_A_page1.json

# Seite 2 (Issues 1001-2000)
curl -u "name@firma.de:Token" \
  "https://firma.atlassian.net/rest/api/2/search?jql=project=ART_A\
&expand=changelog&maxResults=1000&startAt=1000" \
  -o ART_A_page2.json

# Dateien mit dem Helper zusammenfuehren
python -m helper ART_A_page1.json ART_A_page2.json \
  --output ART_A_merged.json
```

Erhöhen Sie startAt schrittweise um 1.000, bis das Ergebnis weniger als 1.000 Issues enthält — dann haben Sie alle Seiten erfasst. Der **Helper** führt die Seiten zusammen und entfernt Duplikate automatisch.

6.6 Vom JSON-Export zur Workflow-Datei

Nach dem Export müssen Sie eine Workflow-Datei erstellen, die Ihre tatsächlichen Jira-Status abbildet. Das folgende Python-Skript extrahiert alle Status-Namen aus der JSON-Datei:

```
import json
data = json.loads(open('ART_A_page1.json').read())
statuses = set()
for issue in data['issues']:
    for h in issue['changelog']['histories']:
        for item in h['items']:
            if item['field'] == 'status':
                statuses.add(item['toString'])
print(sorted(statuses))
```

Aus den gefundenen Status-Namen erstellen Sie eine workflow.txt — ordnen Sie die Status in logischer Reihenfolge an und setzen Sie <First> und <Closed> (Kapitel 3).

7 Häufige Fragen (FAQ)

Was passiert, wenn expand=changelog fehlt?

transform_data findet keine Statusübergänge und gibt für alle Issues leere Zeitwerte aus. Die Excel-Ausgaben enthalten keine nutzbaren Daten. Stellen Sie sicher, dass expand=changelog immer in der URL steht.

Unterschied zwischen Testdaten und echten Daten?

Für Demos und Schulungen sind Testdaten die bessere Wahl — keine Datenschutzbedenken, reproduzierbar und kontrolliert. Für echte Analysen spiegeln echte Daten den tatsächlichen Workflow-Zustand wider und liefern verlässliche Flow-Metriken.

Kann ich mehrere Jira-Projekte zusammenführen?

Ja — entweder über JQL (`jql=project in (ART_A, ART_B)`) für einen einzigen Export, oder mit dem Helper mehrere separate Exporte zusammenführen. Achten Sie darauf, dass die Workflows beider Projekte kompatibel sind.

Wie groß kann die JSON-Datei werden?

Faustregel: ca. 2–5 KB pro Issue (abhängig von der Changelog-Länge). 1.000 Issues entsprechen ca. 2–5 MB. Bei sehr großen Projekten (10.000+ Issues) empfiehlt sich Paginierung und der Einsatz des Helpers.

Ich erhalte den Fehler 'Unauthorized' beim curl-Aufruf.

Prüfen Sie: E-Mail-Adresse und API-Token korrekt? Sind sie durch einen Doppelpunkt getrennt? Hat Ihr Konto Lesezugriff auf das Projekt? Tipp: Einen neuen API-Token generieren, falls der alte abgelaufen ist oder verloren wurde.

8 Glossar

Begriff	Erklärung
ART	Agile Release Train — ein Team aus mehreren Scrum-Teams in SAFe
API	Application Programming Interface — Programmierschnittstelle
API-Token	Sicherheitsschlüssel für den Zugang zur Jira-REST-API; ersetzt das Passwort bei curl-Abfragen
Changelog	Jira-Protokoll aller Statusänderungen eines Issues (expand=changelog)
CFD	Cumulative Flow Diagram — zeigt den Issuebestand je Stage kumuliert über die Zeit
Closed Stage	Die Stage, die ein Issue als abgeschlossen markiert (workflow.txt: <First> bis zur Closed-Stage)
Cycle Time	Zeit von der ersten aktiven Stage (<First>) bis zur Closed-Stage
First Stage	Erste aktive Stage, ab der die Cycle Time zählt (workflow.txt: <First>)
Helper	SituationReport-Modul zum Zusammenführen mehrerer Jira-JSON-Dateien
Issue	Arbeitselement in Jira (Feature, Bug, Enabler, Story, ...)
JQL	Jira Query Language — Abfragesprache für Jira-Suchen, z. B. project=ART_A
JSON	JavaScript Object Notation — Format der Jira-API-Exporte
Seed	Startwert für den Zufallsgenerator — gleicher Seed ergibt immer dieselbe Ausgabe
Stage	Ein Schritt im Workflow (entspricht einer Jira-Spalte oder einem Status-Namen)
Workflow	Geordnete Abfolge von Stages, die ein Issue durchläuft